

Munshi

Team Q&A Study Guide

Every question a judge, mentor or teammate might ask — the product, the demo, the AI, the code, the honest limits. Written so a non-technical teammate can answer confidently, with the deep technical detail right after each simple answer.

Live app • munshi-ochre.vercel.app

Repo • github.com/tripathisarthak457/munshi (private)

Demo logins • **9000000001 / 1111** (Lakshmi, Telugu) • **9000000002 / 2222** (Ramesh, Hindi)
• **9000000000 / 0000** (Mentor desk)

Stack • Go serverless API + React (Vite) + Neon Postgres + OpenAI gpt-4.1-mini, hosted on Vercel (Mumbai)

1. The 30-second answers
2. The basics — for non-technical teammates
3. The demo, step by step
4. Feature by feature
5. How the AI actually works
6. Stopping the AI from making things up
7. Technical architecture
8. Live government data & scraping
9. Security, privacy and money safety
10. Testing and what we verified
11. Honest limitations
12. Hard questions from judges
13. Plain-English glossary

01 The 30-second answers

If you only remember three answers, remember these.

Q What is Munshi, in one sentence?

A Munshi is a conversational assistant for India's smallest businesses that starts by protecting them from UPI fraud and ends by filling in a real government funding application — in Telugu, Hindi, Tamil or English.

Q Why does it matter?

A A street vendor loses ₹450 to a fake payment screenshot and never applies for the ₹50,000 loan she qualifies for, because nobody explains either. Munshi does both in one conversation, using her own daily records as the evidence a bank or scheme officer wants.

Q What makes it different from asking ChatGPT?

A Three things: every fact comes from a tool, not from the model's memory (grant deadlines are only shown if the exact words appear on the official government page); the numbers come from the merchant's own khata records, not from the model; and it remembers the business across sessions and walks a defined journey (safety → QR → khata → blueprint → funding → apply).

The demo line that lands: "Watch the line above each reply — that's Munshi actually calling a tool: *checked against 12 known scam patterns, matched 5 of 27 schemes to your state*. It isn't a chatbot guessing; it's a worker doing steps you can see."

02 The basics — for non-technical teammates

You can answer all of these without touching the code.

Q Who is Munshi for?

A Micro-entrepreneurs: street food carts, kirana shops, tailors, artisans, tea stalls — and first-time founders. People whose business is real but whose paperwork isn't.

Q What does the name mean?

A A *munshi* is the trusted clerk-accountant who traditionally kept a merchant's ledger and wrote their letters and applications. That is exactly the role the app plays.

Q Why does the app look like a cloth-bound ledger?

A Because that is what the user already trusts. The red cloth spine, cream ruled paper, rubber-stamp badges and typewriter numerals come from the *bahi-khata* that sits on every shop counter. It deliberately avoids looking like a generic AI product.

Q Does Munshi touch the merchant's money?

A Never. It generates a standard UPI QR code that points at the merchant's own UPI ID, so payments go bank-to-bank as usual. Munshi holds no money, has no wallet, and never asks for a UPI PIN, OTP, card or account number.

Q What languages does it speak?

A English, Hindi, Telugu and Tamil — the whole interface, the AI's replies, the read-aloud voice and the scheme explanations. You switch language from the top bar; the choice is saved to the account.

Q Do you need to type? Many vendors can't type fast.

A No. You can talk to it — tap the mic and speak in your language — or photograph a handwritten bill and Munshi reads the entries out of the photo. It can also read its answers aloud.

Q Is the data in the demo real?

A Two kinds. The **government scheme and grant data is real and live**, read from official websites. The **three merchant accounts are realistic sample data** we generated — 45 days of item-level sales, purchases, udhaar and chat history — so every screen has something true to show in a two-minute demo.

Q What does it cost to run?

A Almost nothing at demo scale: free Vercel hosting, free Neon Postgres tier, and a few paise of AI usage per conversation because we use a small, fast model (`gpt-4.1-mini`) and cache translations in the database.

Q Does it work on a cheap phone?

A Yes — it is a website, not an app to install, and the phone layout is the primary design. The JavaScript bundle is about 110 KB compressed. Voice uses the phone's own speech engine, so it doesn't need a big download.

Q Does it work offline?

A No. That is a genuine limitation we state openly: the AI, the scheme data and the records all live on the server. Offline entry with later sync is the obvious next step.

03 The demo, step by step

The path we rehearse. Roughly three minutes.

#	DO THIS	SAY THIS
1	Open the landing page, toggle dark mode, switch language to Telugu.	"The counters are live from our database — schemes tracked, scam patterns, when sources were last checked."
2	Log in with one tap as Lakshmi (idli cart, Telugu).	"Her whole business is already in here: 45 days of khata, her cart's stock, past conversations."
3	Paste a scam message: <i>"Accept this request to receive ₹5,000 for your order — enter UPI PIN"</i> .	"Look at the trail: it checked 12 known patterns. SCAM stamp, the warning signs it found, and what to do — in Telugu. Tap the speaker to hear it."
4	Tap the suggestion chip → UPI QR → Print standee .	"A real UPI QR, plus a counter poster that warns her customers about the same scam."
5	Open Overview .	"Masala dosa up 22%, vada down 23%, urad dal runs out in under two days — all computed from her records, no AI guessing."
6	Open Khata → Analyse my sales .	"Now the AI reads those numbers and says what to buy more and less of. The quantities are calculated by us; the AI writes the reasoning."
7	Ask in chat: <i>"Which government schemes help me buy a second cart?"</i>	"It asks only for what it doesn't know — stage and what help she needs — then filters by Andhra Pradesh, and each card links to the official site."
8	Open Blueprint → Flowchart.	"Her plan as a flowchart, with AI-drawn illustrations: today, the investment, the 12-month vision. The financial boxes come from her khata."
9	Switch to Ramesh (Hindi kirana) → Matching → Check eligibility.	"Rule by rule: pass, fail, or unknown, with the scheme's own words quoted. A biotech research call is correctly excluded."
10	Draft application → export DOCX.	"Filled from the blueprint. What it doesn't know is marked NEEDED in yellow — never invented."
11	Log in as Mentor desk .	"Source health, scraper runs, AI latency and token use, support tickets."

Q How do we reset the demo data between judging rounds?

A Run `go run ./cmd/dev reset-demo` from the project folder. It deletes the three demo accounts and regenerates everything — profiles, 45 days of stock and khata, chats, tickets — in about two minutes. Optionally run `node scripts/prime-demo.mjs` to pre-build each persona's AI advice, blueprint and illustrations so nothing has to be generated live.

Q What if the internet is slow during the demo?

A Everything except the AI replies is already stored: the seeded chats, the blueprint, the illustrations and the cached stock advice all load instantly. If a live AI call is slow, the tool trail keeps showing what it's doing, and the fallbacks (browser voice, drawn scenes, cached advice) mean nothing looks broken.

Q How long do the AI features take?

A Chat replies 6–14 seconds (they include real tool calls), stock advice 4–9 s, blueprint 6–8 s, eligibility matching 6–11 s, application draft about 8 s, the three illustrations 20–26 s in parallel.

04 Feature by feature

What each screen does, and the sentence that explains why it exists.

Q Fraud desk — what does it do?

A You paste a suspicious SMS, WhatsApp message or a screenshot (or describe a phone call) and it answers with a verdict — SCAM, SUSPICIOUS or LOOKS SAFE — the patterns it matched, the specific warning signs it found in *your* message, and 2–4 actions, plus the 1930 helpline if money may already be gone.

Q Where do the scam patterns come from?

A A curated catalogue of 12 patterns Indian merchants actually face — the "accept to receive money" collect request, fake payment screenshots, QR-to-receive, screen-sharing apps, KYC-expiry SMS, fake customer care, overpayment refunds, electricity disconnection, task-earning jobs, fake soundbox agents, QR sticker swaps and "digital arrest" calls — each with a link to an RBI, NPCI, PIB or cybercrime.gov.in source. On top of that, a daily job pulls Indian UPI-fraud news and the AI files each headline under one of those patterns, so the list stays current.

Q Shop QR — is it a real UPI QR?

A Yes. We build a standard NPCI-format `upi://pay` link with the merchant's verified UPI ID, their shop name and an optional fixed amount, then render it as a QR image. Any UPI app pays it. The printable standee adds a warning line in the merchant's language and reminds them to scan their own QR once to confirm their name appears.

Q Khata — why does a merchant need it in an app?

A Because the ledger is the evidence. No records means no loan, no scheme, no idea which item earns money. Munshi makes recording almost free: speak it, type it, or photograph the bill — the AI turns it into entries, and you tap Save. Nothing is saved without that tap.

Q What exactly does the khata track?

A Sales, expenses, udhaar given (customers who owe you) and udhaar repaid, with dates, notes and the party's name. It shows sales vs expenses per day, profit, average daily sale, biggest expenses and who owes you how much. When an entry names a stock item, it also moves that item's stock.

Q Stock — what does Munshi do with it?

A Every item has a type (product sold from the shelf, dish cooked fresh, or ingredient), a price, a cost and a stock count. From the movement history we compute: units sold per day over 7 and 30 days, the trend against the previous three weeks, days of stock left, margin, 30-day revenue, and a reorder quantity that covers a week of demand. Each item gets a status: out, critical, low, OK, overstock or not selling.

Q And the AI suggestion feature?

A Tap "Analyse my sales". The server hands the AI only those computed numbers; the AI decides which items to buy more of and less of and writes the reasoning in the merchant's language. The quantities shown are ours, not the model's. A guard rule then enforces common sense: an item that is out or nearly out can never be listed under "buy less", even if its sales look like they're falling — they're falling because the shelf is empty.

Q Business overview — what's on it?

A Today's (or yesterday's) sales, 7-day average with trend, 30-day profit, udhaar outstanding, stock value; then what's running out with days left and a suggested order; what sells, ranked by revenue with per-day run rates and trends; average sales by weekday with the best day highlighted; the 30-day sales chart; items slowing down; and the latest AI buying brief.

Q Blueprint — what is it and who is it for?

A A one-page business plan generated from the profile, the remembered facts, the khata numbers and the stock trends. It's what a bank officer or scheme evaluator asks for: what the business is, the problem, customers, revenue model, current financials, how much money is needed and for what, 12-month milestones, risks with mitigations, and what facts are still missing.

Q Why show the blueprint as a flowchart?

A Because a merchant who reads slowly can still follow arrows and pictures. The flow runs: where you are today → the opportunity → the investment (with a proportional bar of where the money goes) → milestones on a timeline → risks as diamond decision shapes with the response beside them → the 12-month vision. The text document version is still there as a second tab, for the officer who wants prose.

Q What are the pictures in the flowchart?

A AI-generated illustrations, one per stage. First the chat model writes three scene descriptions from the blueprint (the actual cart, city and trade), then an image model draws them in one consistent ledger style. On our current API key no image model is available, so the same chat model draws the scenes as SVG vector art instead — still AI-generated, and the code switches to the real image model automatically if the key gains access.

Q Live funding — what's listed?

A About 46 schemes and calls from three kinds of official source, each card showing the source, when it was last checked, the amount only if the page states it, the deadline only if quoted, and a link to the official page. Closed calls stay visible, marked closed — that honesty is part of the point.

Q Scheme Finder — how is it different from the funding list?

A The list is everything; the finder is *for you*. It asks only for the details it doesn't already know — state, gender, age, social category, business domain, stage and what help you need — as tap-to-answer chips, then recommends 3–6 schemes with why each fits you, what you get, how to apply, what to confirm on the site, plus next steps like getting Udyam registration and which documents to keep ready. State-specific schemes are filtered to your state.

Q Matching — what does the score mean?

A For each shortlisted scheme the AI lists that scheme's own eligibility rules and checks each one against the blueprint: pass, fail, or unknown, with the evidence and the scheme's exact words quoted. A clear fail on a mandatory rule forces "not a fit" and a score under 40. The "why not" section stays open so nothing is hidden.

Q Applications — what do you get?

A A draft with the sections these schemes ask for — applicant details, business description, current financials from the khata, funding requirement and use of funds, expected impact, repayment plan, declaration — plus a cover note and a document checklist ticked against what the profile says the merchant has. Anything unknown appears as a yellow [NEEDED: ...] marker. Export as DOCX or print to PDF; sections are editable in the page.

Q What's in the help desk and settings?

A Twelve FAQs (translated into the user's language on demand), an "ask a question" box answered only from that FAQ knowledge base — it admits when it doesn't know and offers a ticket — and ticket raising. Settings covers name, language, theme, auto read-aloud, PIN change, clearing everything Munshi remembers and deleting the account.

Q What is the Mentor desk for?

A It's the admin view for judges and mentors: counts across the system, health of every data source with the last error, every scraper run with timing, AI usage by kind with average and 90th-percentile latency and token totals, and support tickets. It's also a working merchant account (a sample tea counter) so the merchant features can be demoed from it.

05 How the AI actually works

Explain this well and the technical judges relax.

Q Which AI model do you use, and why that one?

A gpt-4.1-mini from OpenAI, for every AI feature. It's fast and cheap enough for a merchant-facing app, supports tool calling, strict JSON output and image understanding. It is also the only model our hackathon key has access to, which is why we built fallbacks around it rather than depending on bigger models.

Q What is "tool calling", in plain words?

A We give the model a list of things the app can do — check a scam, make a QR, read the khata, search funding — and instead of answering from memory it can say "call *check_scam* with this text". Our Go code runs that function, returns the result, and the model writes the reply around real data. The line of ticks above each reply is that sequence, shown to the user.

Q What tools does the agent have?

A Seventeen: `check_scam`, `check_scam_screenshot`, `list_scam_patterns`, `generate_upi_qr`, `add_records`, `get_records_summary`, `stock_advice`, `remember_fact`, `update_profile`, `build_blueprint`, `scheme_intake`, `find_schemes`, `search_funding`, `match_funding`, `draft_application`, `set_stage` and `suggest_next_step`.

Q What happens if someone asks it something unrelated — cricket scores, or what to cook?

A It declines in one warm sentence and offers the nearest business help, without answering the question. The agent has an explicit scope: fraud and payment safety, UPI and QR, khata, stock and buying, business planning, schemes and applications, and using the app. Anything that touches the business *is* in scope — a brother helping at the counter during Diwali, a festival affecting sales, a supplier dispute — so it stays useful without becoming a general chatbot. It also never gives medical, legal, investment or relationship advice.

Q How does one conversation move from fraud to a funding application?

A Every conversation carries a stage — safety, shop QR, khata, blueprint, funding, apply — shown as tabs. Tools change the stage as a side effect (a QR was generated, so we're at onboarding), and at the end of each turn the agent offers one next step as a tappable chip. If the model forgets to offer one, the server adds a sensible next step based on the last tool used, so the journey never stalls.

Q How does it remember the business between sessions?

A Three layers: a structured **profile** (shop, city, state, documents, goal...), a list of **remembered facts** the agent saves as you talk ("husband grinds the batter at night"), and a rolling **summary** per conversation, refreshed every few turns. All three are injected into the system prompt, along with the khata numbers and stock alerts, so a fresh server process still knows the merchant.

Q Can the user see and delete what it remembers?

A Yes — the Business profile page lists every remembered fact with a delete button, and Settings has "clear all memory" and account deletion. We treat memory as the user's property, not ours.

Q What are "structured outputs" and where do you use them?

A Instead of free text we hand the model a strict JSON schema it must fill: the scam verdict, khata entries parsed from speech, the blueprint, the eligibility checks, the scheme recommendations, the buying advice, the image prompts. It makes the answers machine-checkable, which is what lets us validate and override them.

Q How many AI calls does one chat turn make?

A Usually two to four: one where the model decides to call a tool, one after tool results to write the reply, plus one inside some tools (for instance the scam check or scheme finder is itself a structured AI call). We cap the tool rounds at six to keep costs and latency bounded.

Q Why do replies appear all at once instead of streaming word by word?

A Vercel's Go runtime buffers the response, so token-by-token streaming isn't available there. We compensate by animating the tool trail as the reply arrives, so the user sees progress rather than a dead screen.

Q How is translation handled? The brief suggested IndicTrans2.

A IndicTrans2 is a neural translation model you must host on a machine with real memory — it can't run inside a serverless function. So dynamic content (scheme summaries, scam patterns, FAQs) is translated by the same LLM and cached in a Postgres table keyed by a hash of the text, and the interface strings are hand-written in all four languages. We state the trade-off openly; swapping in an IndicTrans2 service later means changing one function.

Q How do voice input and read-aloud work?

A The browser's own speech recognition and speech synthesis first — they support hi-IN, te-IN and ta-IN in Chrome, cost nothing and respond instantly. If the browser can't, we fall back to server-side transcription and text-to-speech; with our current key those models aren't available, and the app switches silently rather than showing an error.

Q What happens when an AI call fails?

A Each tool returns a failure marker that the model is told to apologise for briefly and offer an alternative; the trail shows a red cross instead of a tick. Page-level features surface a readable message, and cached results (stock advice, blueprint, illustrations) keep the screens populated. Every call is logged with duration and success to the `ai_calls` table, visible in the Mentor desk.

06 Stopping the AI from making things up

This is the section mentors are told to probe. Know it cold.

Q How do you guarantee a grant deadline isn't invented?

A Two layers. When we extract a scheme from an official page, the model must also return the exact sentence that states the deadline; we then check that sentence appears character-for-character in the page text we fetched. If it doesn't, the deadline is discarded and the card says "Deadline not stated on source — open all year or check the link". And the model is forbidden from writing deadlines in chat at all: the card renders them from the database row.

Q Show me that in the code.

A `verifyQuote(page, quote)` in `app/grants.go` — it normalises whitespace and case, requires at least four characters, and does a substring check. It has a unit test with a real quote and a hallucinated one. Amounts go through the same gate.

Q Where else do you override the model?

A Four places. Blueprint financials are replaced server-side with numbers computed from the khata. Stock quantities in the buying advice are computed from run rates, not taken from the model. Eligibility quotes that don't appear in the scheme text are blanked. And any grant ID the model invents is dropped, because we match it against the rows we sent.

Q What stops it from recommending a scheme the merchant clearly can't get?

A The prompt names the classic mistakes (a street-vendor scheme for a shop owner, an artisan scheme for a trader, a research call for a food cart, "new units only" for a 14-year-old shop), state-specific schemes are filtered by the merchant's state before the model sees them, and a single clear rule failure forces "not a fit". We also keep the rejected ones visible with reasons instead of quietly dropping them.

Q Could the fraud checker be wrong?

A It can be, and it says so by design: the verdict has three levels and "suspicious" is used when information is missing, the advice is always safe-by-default (never share a PIN or OTP, check the soundbox, call 1930), and every card carries the helpline. It is a first line of defence, not a legal judgement.

Q What about the application draft — could it invent an Aadhaar number or a turnover figure?

A The prompt forbids invented facts and requires the marker `[NEEDED: description]` instead; the draft view highlights those in yellow and counts them ("3 details needed"). Financial sections are told to use the blueprint's verified figures, which themselves come from the khata.

Q Why should we believe the numbers on the Overview page?

A Because no AI touches them. Run rates, trends, days of stock left, reorder quantities, weekday averages and profit are SQL and Go arithmetic over the merchant's own entries, and the classification logic has a unit test covering out, critical, low, OK, overstock and not-selling cases.

07 Technical architecture

For the technical judge — and so non-technical teammates can name the parts.

Q What's the stack?

A A single Go serverless function serves the whole API on Vercel; the interface is React with Vite and one hand-written CSS design system; data lives in Neon Postgres; AI is OpenAI `gpt-4.1-mini`. No backend framework, no UI kit, no state library.

Q Why Go for a hackathon?

A Fast cold starts, a standard library that covers HTTP routing, HTML parsing, XML feeds and concurrency, and a compiler that catches mistakes before deploy. Running the scrapers, the AI calls and the image generation in parallel is a few lines with goroutines.

Q Why does the function run in Mumbai?

A Because several Indian government portals refuse connections from foreign IP addresses. From Singapore, PM SVANidhi, MUDRA, Stand-Up India and CGTMSE all timed out; from Mumbai (`bom1`) they respond. It cost us a little database latency and bought real data.

Q How is the database organised?

A About twenty tables: `users`, `profiles`, `memories`, `sessions`, `messages`, `ledger_entries`, `stock_items`, `stock_moves`, `qr_codes`, `blueprints`, `blueprint_images`, `grants`, `grant_fits`, `drafts`, `scam_patterns`, `scam_checks`, `scraper_runs`, `ai_calls`, `ai_insights`, `translations`, `faqs`, `tickets`. Money is stored as integer paise, never floats. Schema is one embedded SQL file applied idempotently.

Q How does login work?

A Phone number plus a 4-digit PIN, because that's what merchants already use. The PIN is hashed with bcrypt. The session is a signed cookie — user id, expiry and an HMAC-SHA256 signature over both — set `HttpOnly`, `Secure` and `SameSite=Lax`. There's no OTP because we have no SMS provider; we say so rather than faking it.

Q How does the front end work without a router or component library?

A A 40-line hook over the History API handles routes, a small context provides user, language, theme and toasts, and a `useData` hook does fetch-with-loading. The design system is one CSS file of tokens and components; light and dark are two sets of variables. The result is about 110 KB compressed, and the DOCX exporter is loaded only when someone exports.

Q How do the four languages work in the interface?

A One TypeScript dictionary per language with about 200 keys, typed so a missing key fails the build — that's how we know nothing is untranslated. Language-specific fonts (Noto Sans Telugu, Noto Sans Tamil, Hind) load with the page, and numbers and dates use the browser's locale formatting.

Q How is it deployed?

A Push to GitHub, then `vercel deploy --prod`; the build runs the Vite build and compiles the Go function. Secrets (OpenAI key, session secret, cron secret, database URL) are Vercel environment variables. A daily cron hits `/api/cron/refresh` with a bearer secret to refresh scheme sources and the scam-news feed.

Q Roughly how much code is it?

A About 6,500 lines of Go across 15 files and about 4,500 lines of TypeScript/CSS — small enough that any teammate can read a whole file in one sitting, which was deliberate.

Q What would you change if this had to serve 100,000 merchants?

A Move the scrapers and AI generation into a queue instead of request-time, add per-user rate limits, put Postgres in Mumbai next to the functions with read replicas, cache scheme lists at the edge, store images in object storage rather than Postgres, and add proper phone-OTP authentication.

08 Live government data & scraping

Q What are the three sources?

A (1) **BIRAC calls for proposals** — a real HTML table we parse directly, which gives us genuine deadlines. (2) **Official scheme portals** — PM SVANidhi, MUDRA, Stand-Up India, CGTMSE and Atal Innovation Mission, where the page text is read and the AI extracts schemes under quote verification. (3) A **scheme directory** of well-known central and state schemes with hand-written summaries and official links, whose links are re-checked on every refresh.

Q Why a hand-written directory — isn't that cheating on "live data"?

A It's the honest split. Programme facts like "PM SVANidhi is a collateral-free working-capital loan for street vendors" don't change weekly, and an LLM writing them from memory is exactly the hallucination risk we're avoiding. So those summaries are written by us and carry no amounts or deadlines; anything time-sensitive comes from the live sources, and each directory link is verified to still respond, with the check time shown.

Q How fresh is the data, and how do you show it?

A A daily cron, an admin refresh button, and an automatic refresh when the newest data is over 24 hours old. Every card shows "checked 2 hours ago", and the funding page has a source-health strip with per-source counts and errors.

Q What happens when a source breaks?

A The last good rows stay, the run is logged with the error, and the source shows red. Right now MUDRA's site has a broken TLS certificate chain — we deliberately did *not* disable certificate checking to work around it, because an app that tells people about fraud shouldn't accept unverified connections. It shows as a failing source instead.

Q Why not use the myScheme API?

A We tried; its search API returned server errors for every request we made, from two networks. Rather than fake it, we used sources that actually respond and documented the attempt.

Q Is scraping these sites acceptable?

A We read a handful of public government pages once a day with an identifying user agent, cache the result, and link users to the official page rather than replacing it. No logins, no paywalls, no bulk crawling.

09 Security, privacy and money safety

Q What does Munshi refuse to ask for?

A UPI PIN, OTP, passwords, full Aadhaar and bank account numbers. It's written into the agent's instructions, and if a user volunteers one the assistant is told to warn them instead of storing it.

Q How is the merchant's data protected?

A PINs are bcrypt-hashed; sessions are signed cookies with server-side secrets; every database query is parameterised, so there's no SQL injection surface; every route scopes rows to the logged-in user id, and QR images and blueprint pictures are only served to their owner. Admin-only routes check the role server-side.

Q Can a merchant delete their data?

A Yes. Settings has "clear all memory" and "delete my account", which requires the PIN and cascades to every related row — chats, khata, stock, blueprints, drafts.

Q You let an AI produce SVG images. Isn't that a security risk?

A It would be if we served it blindly. The SVG passes a strict allow-list sanitiser: only shape and gradient elements survive, and anything with a script, event handler, embedded image, external link or stylesheet is rejected outright. It's served with `Content-Security-Policy: default-src 'none'` and `nosniff`, and rendered only inside an `<img>` tag, where scripts can't run anyway. There's a unit test with six malicious payloads.

Q What about uploaded photos and screenshots?

A They're checked to be images, resized in the browser before upload to keep requests small, sent to the AI for the check, and not stored — the conversation keeps a marker that an image was attached, not the image.

Q What security work is still missing?

A Per-user rate limiting, phone-number verification by OTP, an audit log for admin actions, and encryption of the khata at rest beyond what the database provides. We'd rather name these than claim they're done.

10 Testing and what we verified

Q What is actually tested?

A Go unit tests cover the UPI link builder and VPA validation, the BIRAC parser against a saved copy of the real page, quote verification with a genuine and a hallucinated quote, grant open/closed/rolling status, stock classification across all six states, the advice grounding guard, and the SVG sanitiser against six attack payloads.

Q How do you test the whole thing end to end?

A `node scripts/smoke.mjs` runs 15 checks against the live deployment: wrong PIN rejected, login, khata numbers, Hindi voice parsing into correct entries, bad UPI ID rejected, QR creation and PNG, a scam message correctly flagged, grants present with no unquoted deadlines, blueprint financials matching the khata exactly, matching producing rule-by-rule results with the biotech call excluded, a draft with NEEDED markers, the FAQ answer, voice availability, admin access denied to non-admins, and the admin overview.

Q How did you check the interface?

A We drove the deployed app in a browser and inspected the live DOM across ten pages in both light and dark mode at phone width (375 px): no console errors, no horizontal overflow, headings and content present in each language. The scheme-finder flow was completed through the real UI as Lakshmi, and the AI artwork was verified by rendering it to a canvas and measuring drawn pixels.

Q What isn't tested?

A The screenshot fraud check hasn't been run end to end with a real screenshot, there are no automated browser tests, and the seeded sample data — while realistic — is generated, not collected from real shops.

11 Honest limitations

Say these before a judge finds them. It buys credibility.

LIMITATION	WHY, AND WHAT WE'D DO
No AI photo-realistic images	Our API key has access to exactly one model, <code>gpt-4.1-mini</code> . The flowchart art is AI-drawn vector work instead; enabling <code>gpt-image-1</code> on the key switches it over with no code change.
No server voice models	Same key limitation. Voice uses the browser's speech engine, which covers Hindi, Telugu and Tamil in Chrome but not every phone.
No recipe link between dishes and ingredients	Selling 40 idlis doesn't yet deduct rice and dal automatically; ingredient use is tracked separately. A simple per-dish recipe table would close this.
Scheme matching uses tag overlap	We rank candidates by tag overlap before the AI checks rules. Fine for ~50 schemes; embeddings would be needed for thousands.
No OTP verification	Anyone who knows a phone number and PIN can log in. Real deployment needs SMS OTP.
Replies aren't streamed	Vercel's Go runtime buffers responses; the animated tool trail keeps the wait legible.
Two sources currently failing	MUDRA has a broken certificate chain and the Atal Innovation Mission homepage lists no calls. Both show as failing rather than being hidden.
Demo data is synthetic	Realistic and internally consistent — sales derived from item-level history — but not from real shops.
No offline mode	Needed for vendors with patchy data. Local-first entry with sync is the next step.

12 Hard questions from judges

Q "Isn't this just a ChatGPT wrapper?"

A A wrapper passes your question to a model and prints the answer. Here the model can't state a deadline, an amount or a financial figure at all — those are rendered from database rows that passed quote verification, or computed from the merchant's khata. The model's job is judgement and language; the app owns the facts. Remove the AI and you still have a working QR generator, ledger, stock tracker and live scheme database.

Q "How is this different from myScheme or a government portal?"

A A portal answers "what schemes exist". Munshi answers "what can I get, am I eligible, and here is my application half-written" — using records it helped the merchant build, in their language, starting from a problem they feel today (fraud) rather than paperwork they're avoiding.

Q "Why would a vendor trust an app with her ledger?"

A Because she gets something immediately — a scam checked, a QR printed — before being asked for anything. Nothing is saved without her tap, she can see and delete everything Munshi remembers, and the app never asks for a PIN or OTP, which is exactly the behaviour we teach her to expect from anyone legitimate.

Q "What's the business model?"

A Free for merchants. The value is in the aggregate: lenders and scheme facilitators pay for verified, consented application-ready borrower profiles; FMCG distributors pay for demand signals; state missions fund vendor onboarding. None of that requires charging the vendor.

Q "What if the AI gives harmful financial advice?"

A Scope limits it: no investment advice, no interest-rate promises, no eligibility guarantees. Scheme cards push the user to the official page for the rules, matching outputs "unknown" rather than assuming, and the fraud advice is always the conservative action. The riskier the claim, the more the app renders facts instead of letting the model talk.

Q "You had 5 hours. What did you cut?"

A Streaming replies (runtime limitation), offline mode, OTP login, per-dish recipes and automated browser tests. What we refused to cut: quote verification, computed numbers, the four languages, and the honest display of failing data sources.

Q "What would you build next, in order?"

A Offline-first khata entry with sync; SMS OTP login; per-dish recipes so dish sales deduct ingredients; WhatsApp as a second front end, since that's where merchants already live; and a facilitator dashboard so an NGO or bank field officer can help twenty merchants at once.

Q "Show me the single most impressive technical detail."

A Quote verification. The model must return the exact sentence supporting every deadline or amount it extracts, and we check that sentence really exists on the page we fetched — so the system can only repeat what a government site actually says, and says "not stated on source" otherwise. It's twelve lines of Go, has a unit test, and it's the difference between a demo and something a merchant could act on.

13 Plain-English glossary

Enough to follow any technical question in the room.

Q API / endpoint

A The set of web addresses our app's screens call to get or save data, e.g. `/api/records`.

Q Front end / back end

A Front end is what runs in the phone browser; back end is the code on the server.

Q Serverless function

A Server code that starts on demand per request; no machine to run or pay for when idle.

Q LLM

A Large language model — the AI that reads and writes text. Ours is `gpt-4.1-mini`.

Q Prompt

A The instructions we send the model with each request: who it is, the rules, the data.

Q Token

A A chunk of text (roughly $\frac{3}{4}$ of a word). AI usage is billed per token.

Q Tool / function calling

A Letting the model ask our code to do something real instead of guessing the answer.

Q Structured output / JSON schema

A Forcing the model to answer in a fixed data shape so we can check it.

Q Hallucination

A When a model states something plausible but untrue — the risk this project is built to contain.

Q Scraping

A Reading a web page with code to pull out facts like titles and dates.

Q Cron job

A A task that runs automatically on a schedule; ours refreshes scheme data daily.

Q Postgres

A The database. Neon is a hosted Postgres service.

Q bcrypt / HMAC

A Standard ways to store a PIN safely and to sign a login cookie so it can't be forged.

Q SVG

A An image described with shapes and coordinates instead of pixels, so it stays sharp at any size.

Q UPI VPA

A A UPI ID like `name@okaxis` — the address money is sent to.

Q Khata / udhaar

A The shop ledger, and goods given on credit that a customer still owes.

Q Run rate

A How much of an item sells per day on average — the basis of every stock alert.

Q Udyam registration

A Free government MSME registration; the gateway to most schemes and priority-sector loans.